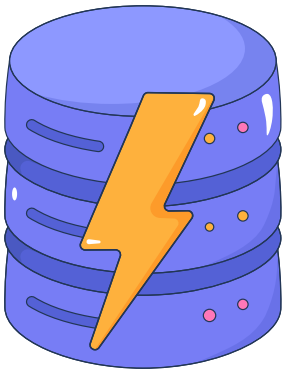
Caching and Caching Invalidation Strategies

Explore various caching techniques including TTL, write-through, read-through, and lazy loading to improve AWS application performance. Understand cache invalidation methods like time-based, manual, event-based, and group-based invalidation, and learn eviction policies such as LRU, LFU, FIFO, and random replacement to manage cache effectively.

Performance optimization is a constant challenge for developers, and caching is one of the most powerful techniques they can apply. By temporarily storing data closer to where it's needed, caching reduces load times, minimizes expensive operations, and improves user experiences.



Common caching strategies

Time To Live (TTL) is a caching strategy that defines how long a piece of data should remain in the cache before it expires automatically. Once the TTL p elapses, the cached data is considered stale and removed or replaced the next

Ask

time it's requested. At that point, a fresh copy is fetched from the source and re-cached with a new TTL.

TTL is ideal for scenarios where data goes stale after a specific period of time. For example, in a user authentication system, session tokens are often cached with a Time To Live (TTL) value. When a user logs into a web application, an access token is generated and cached with a TTL of 30 minutes. This means the user remains authenticated and can make API requests for 30 minutes without re-validation. Once the TTL expires, the token is automatically removed from the cache, and the user must re-authenticate or refresh the token.

TTL automates the lifecycle management of data in the cache. Developers don't need to write custom logic to invalidate stale data. The cache system handles expiry based on a timer. This reduces complexity and prevents memory bloat from unused or outdated entries.

Unlike write-through or read-through strategies that aim for strong consistency by synchronizing cache with every read or write, TTL prioritizes freshness over precision.

Write-through caching

In write-through caching, every time we write (update) data, the system writes it to both the cache and the backing store, a database, simultaneously. A proactive caching approach ensures the data is always readily available.

Pros	Cons
The cache and database are always in sync, and the performance of the database is optimal because of fewer read requests.	It gives slightly slower writes b

Ask

> **Note:** It is ideal for applications where data consistency is crucial, for example, user settings, financial apps.

Read-through caching

In read-through caching, the application first checks the cache. If the data isn't there, that is a cache miss; it fetches from the database and updates the cache for future requests.

Pros	Cons
Automatic cache population	Slow first read (if ca

Read-through caching centralizes cache update mechanisms within the cache layer itself. This makes it ideal for applications that require frequent synchronization with the database while minimizing manual cache management. In simple terms, read-through cache seamlessly plugs into the application.

Lazy loading (or cache-aside)

Lazy loading, also called cache-aside, is similar to read-through, but instead of the cache handling the fetch logic, the application manages it manually.

Lazy load caching provides developers with fine-grained control over what data gets cached and when it is refreshed. This makes it ideal for read-heavy applications where managing cache size is important.

Consider an e-commerce website where only the product listings are load when a user first visits the page. The detailed information, such as images, descriptions, and customer reviews, is fetched only when the user clicks on a

Ask

specific product. This lazy loading approach helps reduce initial page load time and optimizes resource usage.

Pros	Cons
It reduces unnecessary memory usage by giving developers fine-grained control over cached data.	Application logic for cache management ca get complex.

Cache invalidation

Caching boosts performance, but outdated data can cause inconsistencies, stale results, or even security issues. This is where cache invalidation comes in. It is the process of removing or updating entries in the cache that are no longer valid.

Effective invalidation ensures the cache reflects accurate, up-to-date information. It's especially important when underlying data changes frequently or when precision is critical.

Types of cache invalidation

There are many types of cache invalidation techniques developers can imp

Ask

- **Time-based invalidation:** Data is automatically removed from the cache after a specified duration. This method reduces the need for manual validation and is ideal for time-sensitive data. TTL uses time-based invalidation strategy.
- **Manual invalidation:** Developers manually remove specific cache entries after a known update. This works well in lazy loading and cache-aside strategies where the application has control.
- **Event-based invalidation:** The cache is updated or cleared based on events, for example, data change triggers, pub/sub mechanisms. This is useful in distributed systems or microservices.
- **Group-based invalidation:** Related cache entries are grouped under a common tag or namespace. When a change occurs, the entire group can be invalidated in one operation. This strategy is especially useful for entities that share dependencies, such as products within a category or blog posts by a specific author.

> **Tip:** Always design the cache invalidation policies carefully to avoid serving stale or inconsistent data.

Cache invalidation strategies

When a cache reaches its predefined capacity, an invalidation strategy is triggered to decide which entries to evict. The following strategies represent common approaches developers use to manage cache contents and ensure data integrity.

- **Least recently used (LRU):** When the cache reaches capacity, the least recently used entries are evicted first. LRU assumes that items not accessed recently are less likely to be needed again soon. This strategy is often used in operating systems to manage memory pages.
- **Least frequently used (LFU):** This strategy removes the cache entries that have been accessed the fewest number of times. It's useful for eliminating

Ask

rarely used data and is employed in systems like search engines to prioritize frequently accessed queries.

- **Random replacement:** In this approach, the system randomly selects a cache entry to evict when space is needed. Though not optimized, it doesn't require tracking or metadata, and is sometimes used in load balancers or basic memory managers.
- **First-In-First-Out (FIFO):** FIFO removes the oldest cache entries first, assuming they are least likely to be useful now. It is a simple and predictable method, common in queue-based systems like printer job management.

Conclusion

Selecting the right caching strategy can significantly enhance application performance and user satisfaction. By understanding when and how to apply each method, developers can ensure a balance between speed, consistency, and system resource management.

Ask